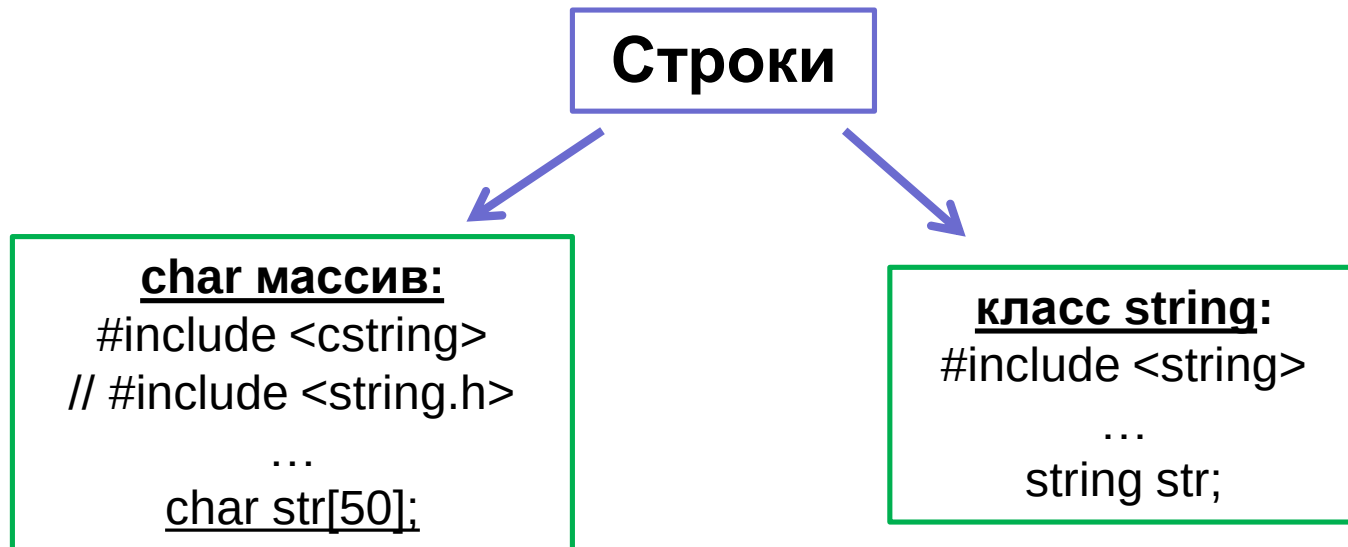


Лекция 3

Строки. Работа с памятью.

1 Строки в языке C++



Конструкторы класса string

- 1) `string()` – конструктор без параметров (создает пустую строку);
- 2) `string(string & S)` – конструктор копирования (копия строки `S`);
- 3) `string(const char* S)` – конструктор, принимающий указатель на строку СИМВОЛОВ;
- 4) `string(size_t n, char c)` - повторение символа `c` заданное число `n` раз;
- 5) `string(string & S, size_t start, size_t len)` - строка, содержащая не более, чем `len` символов данной строки `S`, начиная с символа номер `start`.

Как вызвать каждый конструктор?

Операторы класса string

= – присваивание

+ – конкатенация (объединение строк)

+= – присваивание с конкатенацией

== – равенство

!= – неравенство

< – меньше

<= – меньше или равно

> – больше

>= – больше или равно

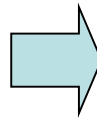
[] – индексация

```
16 string s1 = "s-1";
17 string s2 = "s-2";
18 string s3;
19 bool b;
20
21 // операция '=' (присваивание строк)
22 s3 = s1; // s3 = "s-1"
23
24 // операция '+' - конкатенация строк
25 s3 = s3 + s2; // s3 = "s-1s-2"
26
27 // операция '+= ' - присваивание с конкатенацией
28 s3 = "s-3";
29 s3 += "abc"; // s3 = "s-3abc"
30
31 // операция '==' - сравнение строк
32 b = s2==s1; // b = false
33 b = s2=="s-2"; // b = true
34
35 // операция '!=' - сравнение строк (не равно)
36 s1 = "s1";
37 s2 = "s2";
38 b = s1 != s2; // b = true
```

```
39
40 // операции '<' и '>' - сравнение строк
41 s1 = "abcd";
42 s2 = "de";
43 b = s1 > s2; // b = false
44 b = s1 < s2; // b = true
45
46 // операции '<=' и '>=' - сравнение строк
47 s1 = "abcd";
48 s2 = "ab";
49 b = s1 >= s2; // b = true
50 b = s1 <= s2; // b = false
51 b = s2 >= "ab"; // b = true
52
53 // операция '[' - индексация
54 char c;
55 s1 = "abcd";
56 c = s1[2]; // c = 'c'
57 c = s1[0]; // c = 'a'
```

Ввод/вывод строк

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main()
7  {
8      cout << "Введите слово: ";
9      string str;
10     cin >> str;
11     cout << str << endl;
12
13     cout << "Введите два слова: ";
14     string str1;
15     cin >> str1;
16     cout << str1 << endl;
17
18     cin.ignore(10, '\n');
19
20     cout << "Введите два слова: ";
21     string str2;
22     getline(cin, str2);
23     cout << str2 << endl;
24 }
```



```
Введите слово: один
один
Введите два слова: два три
два
Введите два слова: четыре пять
четыре пять
```

```

1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main()
7  {
8      cout << "Введите число: ";
9      int choice;
10     cin >> choice;
11
12     cout << "Введите имя: ";
13     string myName;
14     getline(cin, myName);
15
16     cout << "Привет, " << myName <<
17     "; число: " << choice << '\n';
18     return 0;
19 }

```



```

Введите число: 55
Введите имя: Привет, ; число: 55

```

```

1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main()
7  {
8      cout << "Введите число: ";
9      int choice;
10     cin >> choice;
11
12     // игнорируем до 32767 символов
13     // до тех пор, пока \n не будет
14     // удалён из потока
15     cin.ignore(32767, '\n');
16
17     cout << "Введите имя: ";
18     string myName;
19     getline(cin, myName);
20
21     cout << "Привет, " << myName <<
22     "; число: " << choice << '\n';
23     return 0;
24 }

```



```

Введите число: 55
Введите имя: Olga
Привет, Olga; число: 55

```

Некоторые методы класса string

<u>assign</u>	назначает символы строке
<u>size</u> <u>length</u>	возвращает количество символов в строке
<u>clear</u>	очищает содержимое строки
<u>insert</u>	вставка символов
<u>erase</u>	удаление символов
<u>push_back</u>	добавление символа в конец строки
<u>pop_back</u>	удаляет последний символ
<u>append</u>	добавляет символы в конец строки
<u>compare</u>	сравнивает две строки
<u>replace</u>	заменяет каждое вхождение указанного символа
<u>substr</u>	возвращает подстроку
<u>copy</u>	копирует символы
<u>find</u>	поиск символов в строке
<u>rfind</u>	поиск последнего вхождения подстроки
<u>find first of</u>	поиск первого вхождения символов
<u>find first not of</u>	найти первое вхождение отсутствия символов
<u>find last of</u>	найти последнее вхождение символов
<u>find last not of</u>	найти последнее вхождение отсутствия символов

2 Управление памятью

Выделение памяти:

1) под единичный объект определенного типа:

```
int *pint = new int(1024);
```

2) под массив заданного размера, состоящий из элементов определенного типа:

```
int *pia = new int[4];
```

Освобождение памяти:

1) единичного объекта :

```
delete pint;
```

2) массива:

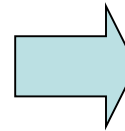
```
delete[] pia;
```

Перераспределение
выделенной
памяти?


```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class People
6  {
7      string FIO;
8      int AGE;
9
10     public:
11         void setPeople(string fio, int age)
12         {
13             FIO = fio;
14             AGE = age;
15         }
16
17         void showPeople()
18         {
19             cout << "ФИО: " << FIO << endl;
20             cout << "Возраст: " << AGE << endl;
21         }
22     };
23
24     int main()
25     {
26         People P[3];
27         P[0].setPeople("Петров П.П.", 20);
28         P[1].setPeople("Сидоров С.С.", 30);
29         P[2].setPeople("Иванов И.И.", 40);
30         P[0].showPeople();
31         P[1].showPeople();
32         P[2].showPeople();
33     }

```



```

ФИО: Петров П.П.
Возраст: 20
ФИО: Сидоров С.С.
Возраст: 30
ФИО: Иванов И.И.
Возраст: 40

```

```

1  #include <iostream>
2  #include <string>
3  #include <cstdlib>
4  using namespace std;
5
6  class People
7  {
8      string FIO;
9      int AGE;
10
11  public:
12      void setPeople(string fio, int age)
13      {
14          FIO = fio;
15          AGE = age;
16      }
17
18      void showPeople()
19      {
20          cout << "ФИО: " << FIO << endl;
21          cout << "Возраст: " << AGE << endl;
22      }
23  };
24
25  int main()
26  {
27      People *P = (People*)malloc(3*sizeof(People));
28      (P+0)->setPeople("Петров П.П.", 20);
29      (P+1)->setPeople("Сидоров С.С.", 30);
30      (P+2)->setPeople("Иванов И.И.", 40);
31      (P+0)->showPeople();
32      (P+1)->showPeople();
33      (P+2)->showPeople();
34  }

```

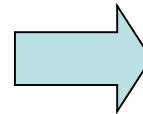


ОШИБКА

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class People
6  {
7      string FIO;
8      int AGE;
9
10     public:
11         void setPeople(string fio, int age)
12         {
13             FIO = fio;
14             AGE = age;
15         }
16
17         void showPeople()
18         {
19             cout << "ФИО: " << FIO << endl;
20             cout << "Возраст: " << AGE << endl;
21         }
22     };
23
24     int main()
25     {
26         People *P = new People[3];
27         (P+0)->setPeople("Петров П.П.", 20);
28         (P+1)->setPeople("Сидоров С.С.", 30);
29         (P+2)->setPeople("Иванов И.И.", 40);
30         (P+0)->showPeople();
31         (P+1)->showPeople();
32         (P+2)->showPeople();
33     }

```



```

ФИО: Петров П.П.
Возраст: 20
ФИО: Сидоров С.С.
Возраст: 30
ФИО: Иванов И.И.
Возраст: 40

```

```

1  #include <iostream>
2  #include <string>
3  #include <cstdlib>
4  using namespace std;
5
6  class People
7  {
8      string *FIO;
9      int AGE;
10
11  public:
12      void setPeople(string *fio, int age)
13      {
14          FIO = fio;
15          AGE = age;
16      }
17
18      void showPeople()
19      {
20          cout << "ФИО: " << *FIO << endl;
21          cout << "Возраст: " << AGE << endl;
22      }
23 };

```

```

23 },
24
25 int main()
26 {
27     People *P = (People*)malloc(3*sizeof(People));
28     string fio[3];
29
30     fio[0] = "Петров П.П.";
31     (P+0)->setPeople(&fio[0], 20);
32
33     fio[1] = "Сидоров С.С.";
34     (P+1)->setPeople(&fio[1], 30);
35
36     fio[2] = "Иванов И.И.";
37     (P+2)->setPeople(&fio[2], 40);
38
39     (P+0)->showPeople();
40     (P+1)->showPeople();
41     (P+2)->showPeople();
42 }

```



```

ФИО: Петров П.П.
Возраст: 20
ФИО: Сидоров С.С.
Возраст: 30
ФИО: Иванов И.И.
Возраст: 40

```



```

24 int main()
25 {
26     People *P = new People[2];
27
28     (P+0)->setPeople("Петров П.П.", 20);
29     (P+1)->setPeople("Сидоров С.С.", 30);
30     for (int i = 0; i<2; i++)
31     { (P+i)->showPeople(); }
32
33     People *P2 = new People[3];
34     for (int i = 0; i<2; i++)
35     { P2[i] = P[i]; }
36     delete[] P;
37     P = P2;
38
39     (P+2)->setPeople("Иванов И.И.", 40);
40     cout << endl;
41     for (int i = 0; i<3; i++)
42     { (P+i)->showPeople(); }
43
44     P2 = new People[4];
45     for (int i = 0; i<3; i++)
46     { P2[i] = P[i]; }
47     delete[] P;
48     P = P2;
49
50     cout << endl;
51
52     (P+3)->setPeople("Макаров М.М.", 40);
53     for (int i = 0; i<4; i++)
54     { (P+i)->showPeople(); }
55 }

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class People
6  {
7      string FIO;
8      int AGE;
9
10     public:
11         void setPeople(string fio, int age)
12         {
13             FIO = fio;
14             AGE = age;
15         }
16
17         void showPeople()
18         {
19             cout << "ФИО: " << FIO << endl;
20             cout << "Возраст: " << AGE << endl;
21         }
22     };

```

```

ФИО: Петров П.П.
Возраст: 20
ФИО: Сидоров С.С.
Возраст: 30
ФИО: Петров П.П.
Возраст: 20
ФИО: Сидоров С.С.
Возраст: 30
ФИО: Иванов И.И.
Возраст: 40
ФИО: Петров П.П.
Возраст: 20
ФИО: Сидоров С.С.
Возраст: 30
ФИО: Иванов И.И.
Возраст: 40
ФИО: Макаров М.М.
Возраст: 40

```